Now that Twig is ready to do things, let's have it generate some markup for us:

```
print $twig->render('variables.html.twig', [
    'cat' => 'Jess',
    'set' => 'This is not a good idea.'
]);
```

… note we are not setting a `cache` config value when we create the Twig environment object, avoiding activating Twig's caching features for now. Unless otherwise stated, all remaining Twig examples in this section do not use caching and assume you're using the same autoloader and Twig instantiation boilerplate. With that done, add the following `variables.html.twig` template to your `views` directory:

```
{# views/variables.html.twig #}
{% set displayCat = cat ~ ' the Cat' %}
<p>{{ displayCat }} says "meow".</p>
<p>{{ set }}</p>
```

Now you can either run the new `index.php` file directly from the command line or request it from a `cli-server` instance with curl:

```
$ curl http://localhost:8080/index.php
<p>Jess the Cat says "meow".</p>
<p>This is not a good idea.</p>
```

The `{% set %}` tag allows us to create a new variable `displayCat`, which can then be used in output tags as `{{ displayCat }}` – if `displayCat` was already defined, then Twig simply overwrites it with the new value. Here we've used a string concatenation operator `~` to build a new string, and we'll be looking in detail at this and many others later in this section. You'll notice I created a `set` variable in the `render()` call, but the Twig engine is clever enough to know that `{% set ... %}` is meant to be a variable setting call, and `{{ set }}` should write out the data assigned to the template when we called `render()`. It's a good feature to save yourself from blowing up your templates, but it's better to avoid the problem in the first place. If you find you just can't avoid colliding with an existing word in the Twig language (such as `if`, `for`, `cache`, `block`… there are quite a few), use the following trick, which also serves as a neat demo for accessing structured data: